

THE UNIVERSITY OF MELBOURNE

COMP90038 Algorithms and Complexity

Lecture 9: Decrease-and-Conquer-by-a-Constant

Olya Ohrimenko

(with thanks to Harald Søndergaard)

Assignment 1 is out

See LMS for consultation hours

- In person and on Zoom
- Ed Discussion board (do not post answers)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Consultation hours

Tuesdays 2-4pm in person, CIS Student Hub

Thursdays 10-11 am Zoom, see Canvas for a link

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Decrease-and-Conquer-by-a-Constant

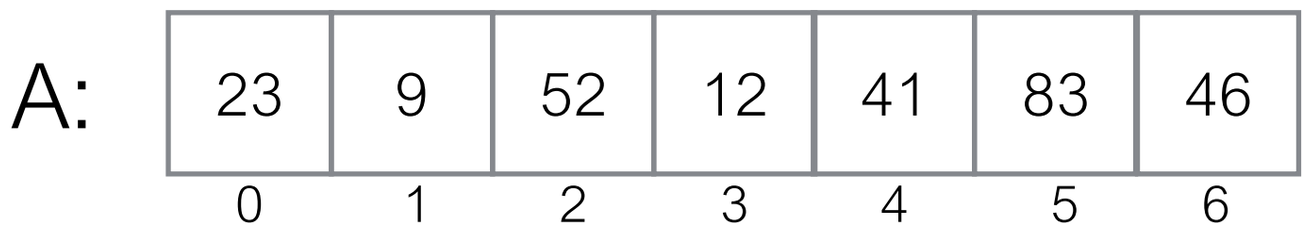
- In this approach, the size of the problem is reduced by some **constant** in each iteration of the algorithm.

- A simple example is the following approach to sorting:
  To sort an array of length n, just

  1. sort the first n – 1 items, then
  2. locate the cell that should hold the last item, shift all elements to its right to the right, and place the last element.

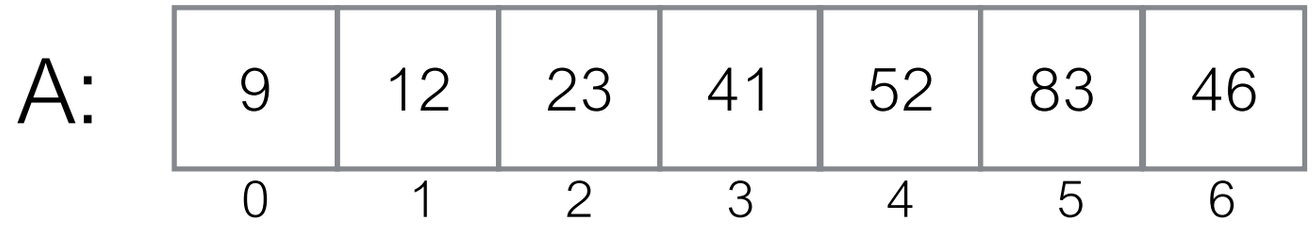
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Sorting n items



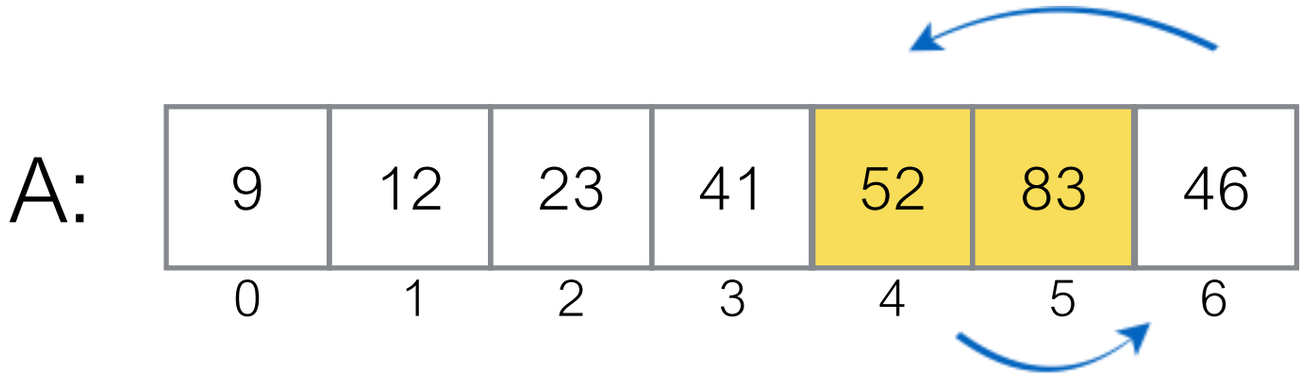
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Intuition



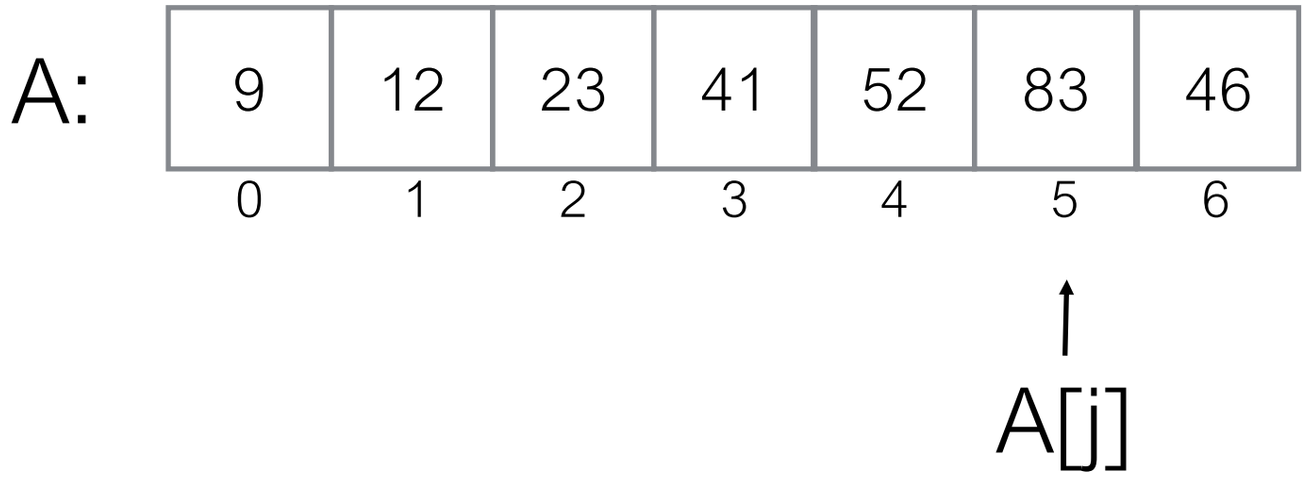
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Intuition



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

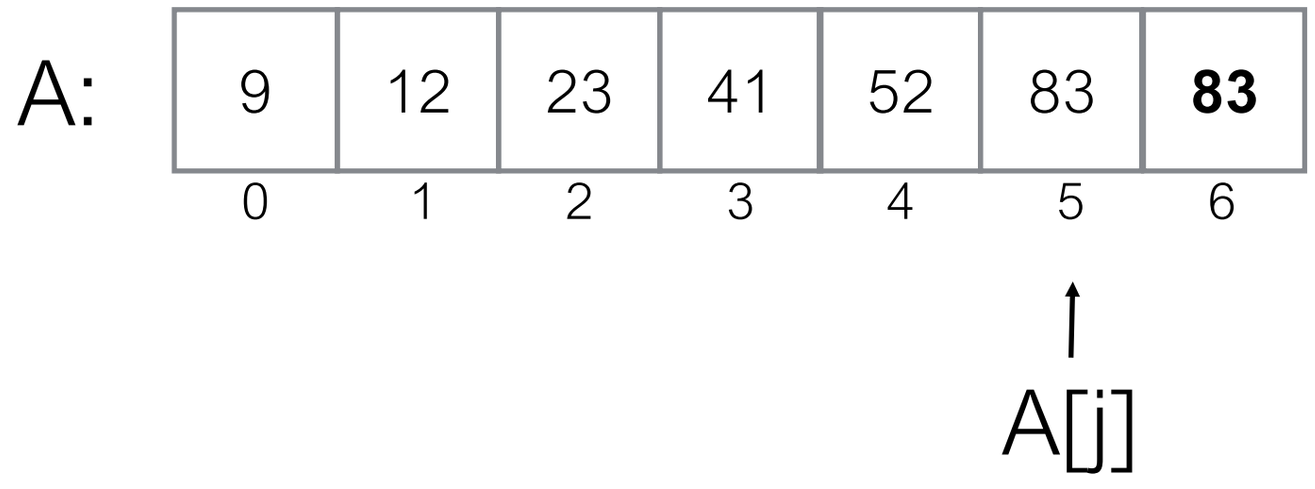
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

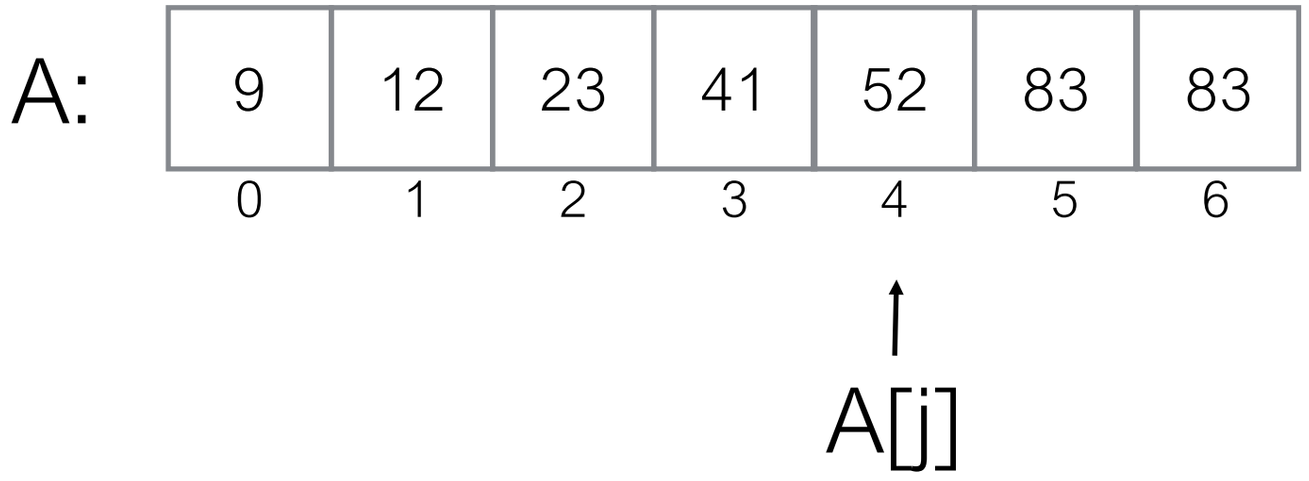
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

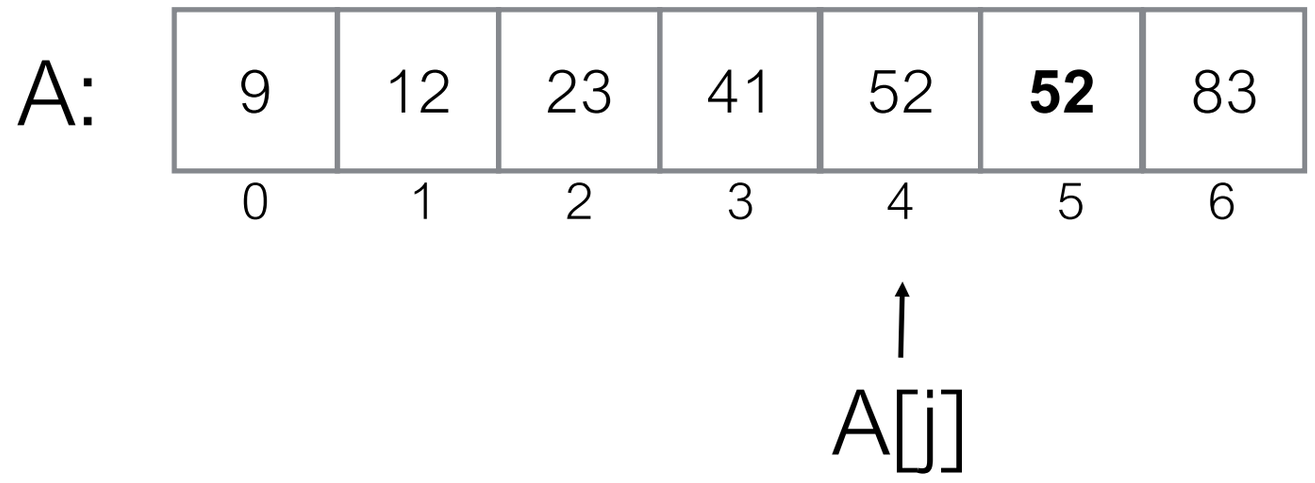
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

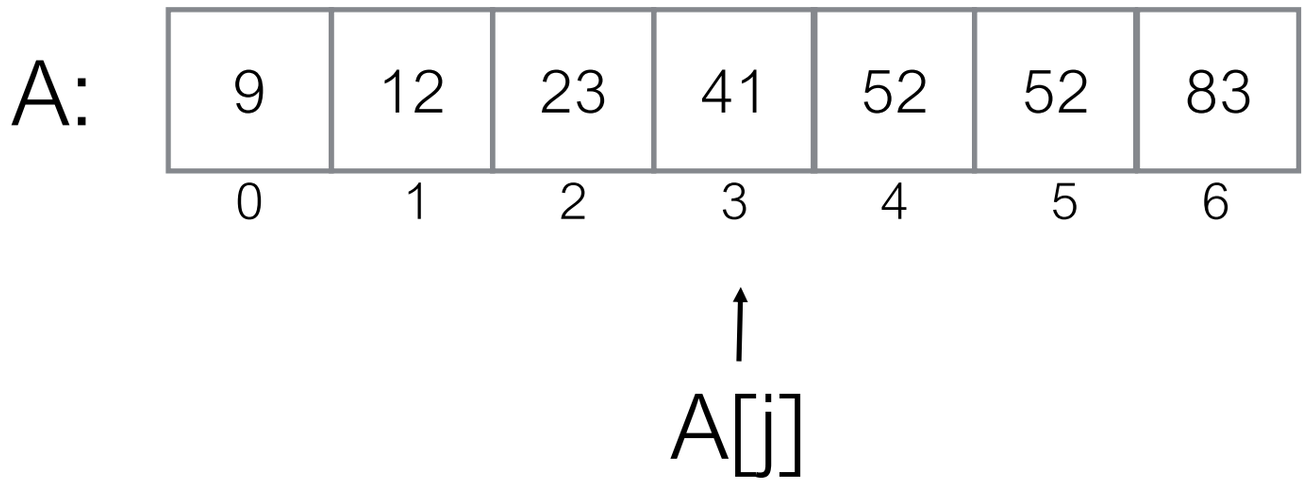
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

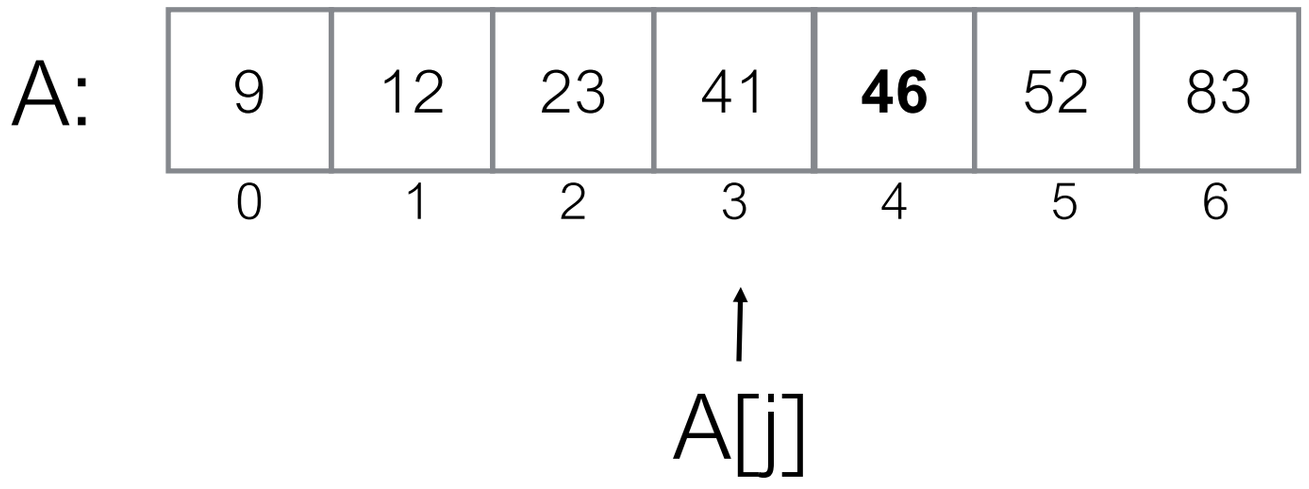
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

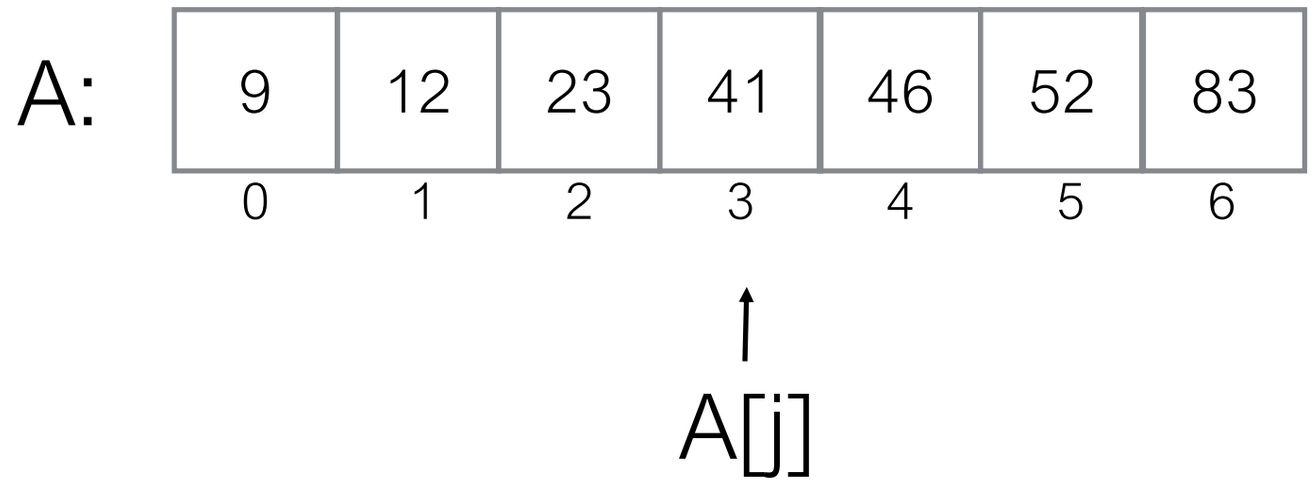
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

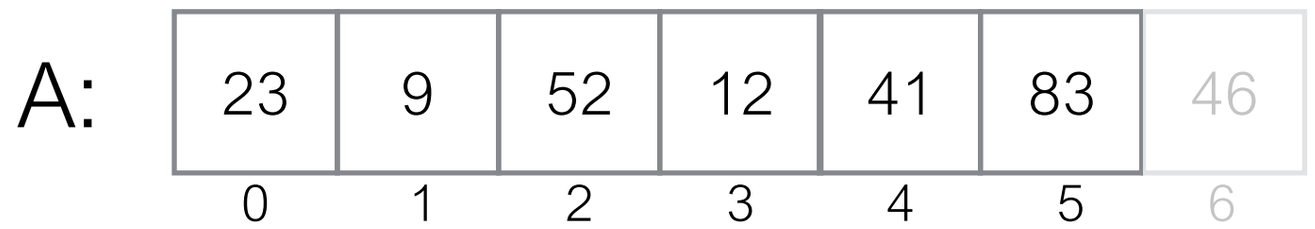
Insertion Sort Intuition



v: 46

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Sorting n items



Sort first n-1 items

Insert nth item in the correct position

To sort A[0] .. A[i]:

First sort A[0] .. A[i-1]

Then insert A[i] in its proper place

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Example

THE UNIVERSITY OF MELBOURNE

A:

23	9	52	12	41	83	46
0	1	2	3	4	5	6

Sort A[0], A[1]

23	9
0	1

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Example



THE UNIVERSITY OF MELBOURNE

A:

23	9	52	12	41	83	46
0	1	2	3	4	5	6

Sort A[0], A[1]

9	23
0	1

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Example



THE UNIVERSITY OF MELBOURNE

A:

0	1	2	3	4	5	6
23	9	52	12	41	83	46

Sort A[0], A[1]

0	1
9	23

Sort A[0]..A[3]

0	1	2	3
9	23	52	12

0	1	2	3
9	12	23	52

Sort A[0]..A[2]

0	1	2
9	23	52

.....

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort

- Sorting an array A[0]..A[n – 1]:

- To sort A[0] .. A[i] first sort A[0] .. A[i-1], then insert A[i] in its proper place

function INSERTIONSORT($A[\cdot], n$)
  for $i \leftarrow 1$ **to** $n - 1$ **do**
    $v \leftarrow A[i]$
    $j \leftarrow i - 1$
    while $j \geq 0$ **and** $v < A[j]$ **do**
      $A[j + 1] \leftarrow A[j]$
      $j \leftarrow j - 1$
    $A[j + 1] \leftarrow v$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Complexity of Insertion Sort



THE UNIVERSITY OF MELBOURNE

- The for loop is traversed n – 1 times. In the ith round, the test v < A[j] is performed i times, in the worst case.
- Hence the worst-case running time is

$$\sum_{i=1}^{n-1}\sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2}$$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

What input is worst&best for insertion sort?

THE UNIVERSITY OF MELBOURNE

a

9	12	23	41	52	46	83

b

83	52	46	41	23	12	9

```
function InsertionSort(A[·], n)
    for i ← 1 to n − 1 do
        v ← A[i]
        j ← i − 1
        while j ≥ 0 and v < A[j] do
            A[j + 1] ← A[j]
            j ← j − 1
        A[j + 1] ← v
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Trick of Posting a Sentinel



THE UNIVERSITY OF MELBOURNE

- If we are sorting elements from a domain that is bounded from below, that is, there is a minimal element min, and the array A was known to have a free cell to the left of A[0], then we could simplify the test. Namely, we would place min (a sentinel) in that cell (A[−1]) and change the test from

$$j \geq 0 \text{ and } v < A[j]$$

  to just

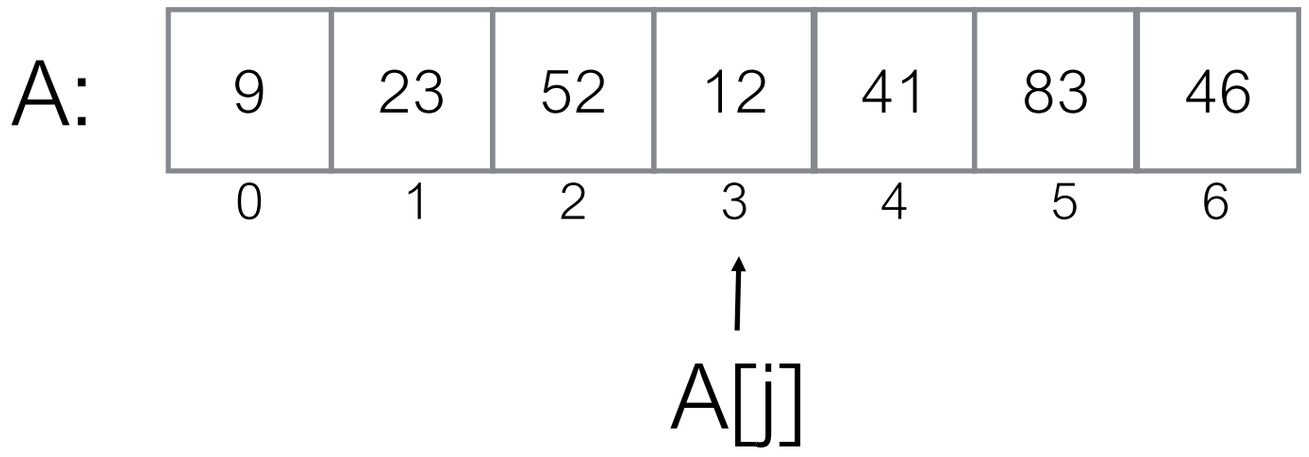
$$v < A[j]$$

- That will speed up insertion sort by a constant factor.

- For this reason, extreme array cells (such as A[0] in C, and/or A[n + 1]) are sometimes left free deliberately, so that they can be used to hold sentinels; only A[1] to A[n] hold proper data.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

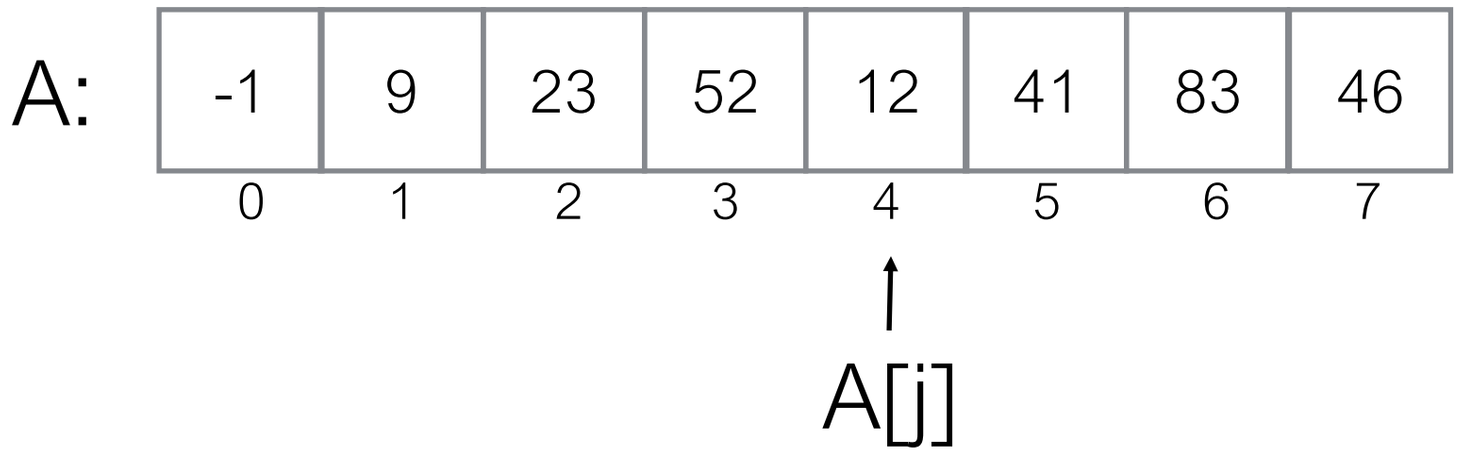
Posting a Sentinel



Test required: $j \geq 0$ and $v < A[j]$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Posting a Sentinel



Test required: v < A[j]

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Insertion Sort

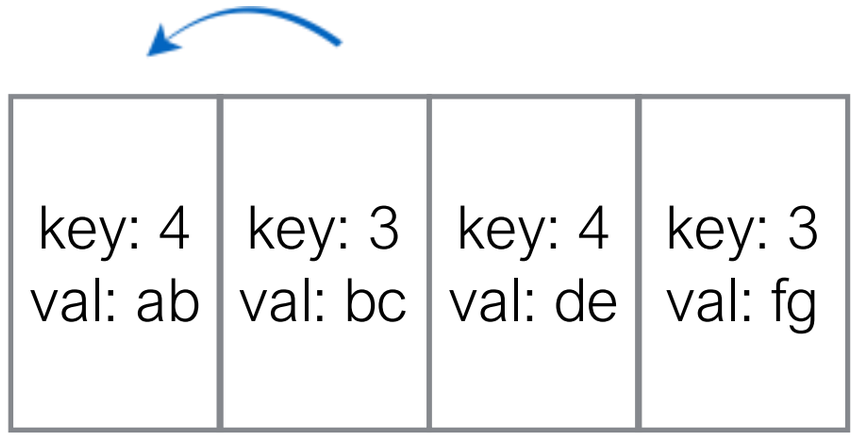


THE UNIVERSITY OF MELBOURNE

- Easy to understand and implement.
- Average-case and worst-case complexity both quadratic.
- However, linear for almost-sorted input.
- Some cleverer sorting algorithms perform almost-sorting and then let insertion sort take over.
- Very good for small arrays (say, a couple of hundred elements).
- In-place? yes
- Stable? ?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

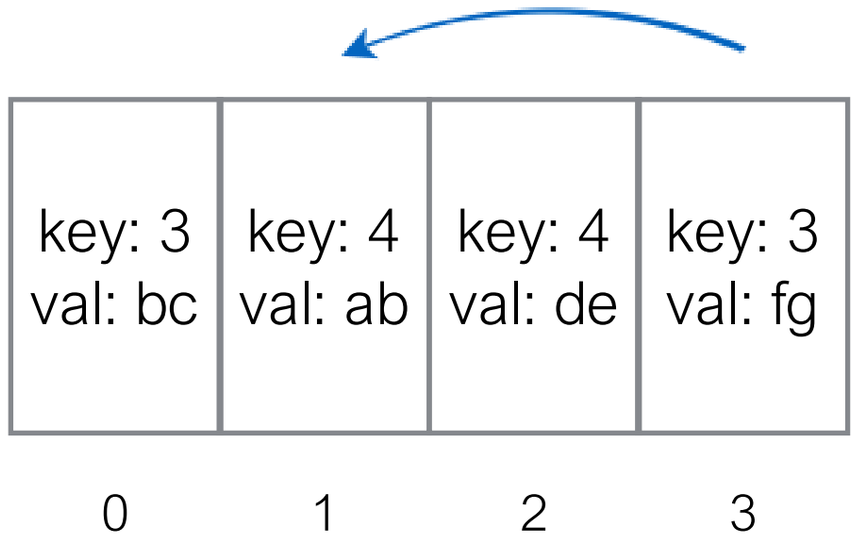
Insertion Sort Stability



function INSERTIONSORT($A[\cdot], n$)
  for $i \leftarrow 1$ **to** $n-1$ **do**
$v \leftarrow A[i]$
$j \leftarrow i-1$
while $j \geq 0$ **and** $v < A[j]$ **do**
    $A[j+1] \leftarrow A[j]$
    $j \leftarrow j-1$
$A[j+1] \leftarrow v$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

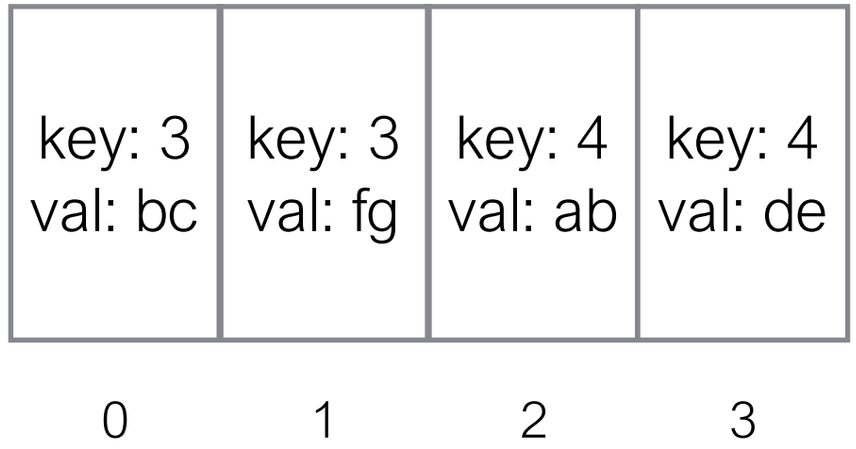
Insertion Sort Stability



function INSERTIONSORT($A[\cdot], n$)
  for $i \leftarrow 1$ **to** $n-1$ **do**
$v \leftarrow A[i]$
$j \leftarrow i-1$
while $j \geq 0$ **and** $v < A[j]$ **do**
    $A[j+1] \leftarrow A[j]$
    $j \leftarrow j-1$
$A[j+1] \leftarrow v$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Insertion Sort Stability



Stable

function INSERTIONSORT($A[\cdot], n$)
  for $i \leftarrow 1$ **to** $n-1$ **do**
    $v \leftarrow A[i]$
    $j \leftarrow i-1$
    while $j \geq 0$ **and** $v < A[j]$ **do**
      $A[j+1] \leftarrow A[j]$
      $j \leftarrow j-1$
    $A[j+1] \leftarrow v$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Insertion Sort

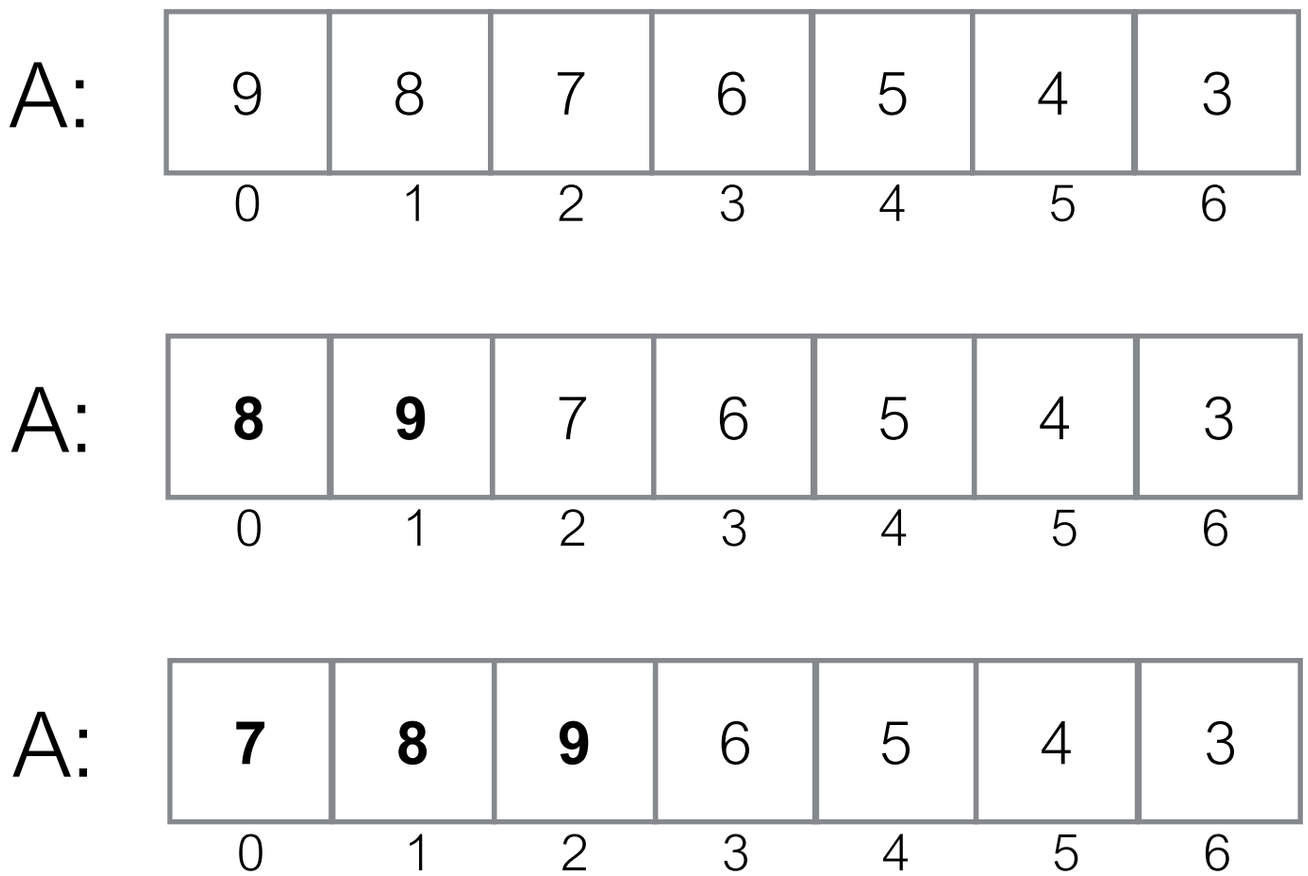


THE UNIVERSITY OF MELBOURNE

- Easy to understand and implement.
- Average-case and worst-case complexity both quadratic.
- However, linear for almost-sorted input.
- Some cleverer sorting algorithms perform almost-sorting and then let insertion sort take over.
- Very good for small arrays (say, a couple of hundred elements).
- In-place? yes
- Stable? yes

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Shellsort: Motivation



It would be better if we could move the 9, 8, etc. to the right faster

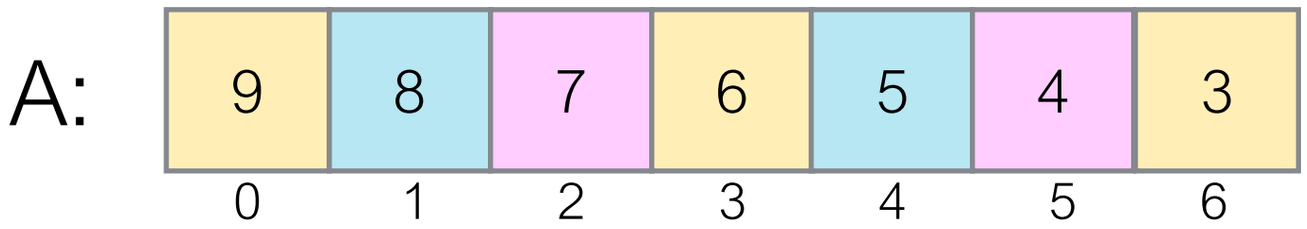
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

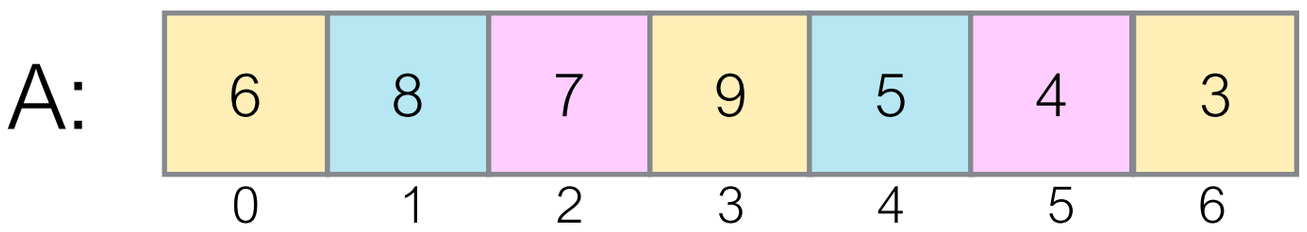
Shellsort: Motivation

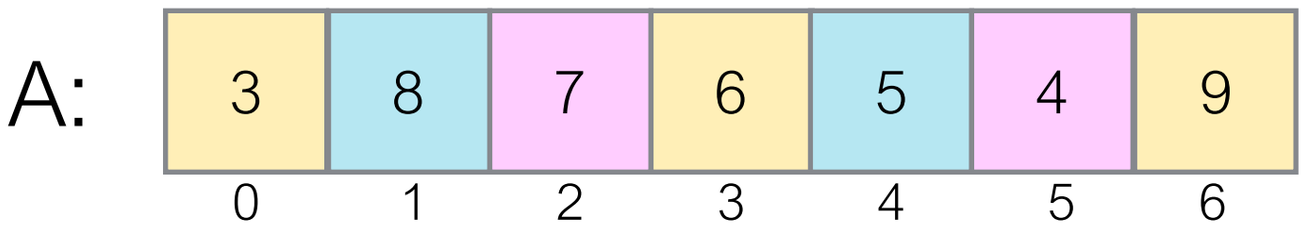


THE UNIVERSITY OF MELBOURNE

Sort the yellow entries







Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Shellsort: Motivation

Sort the blue entries

A:

3	8	7	6	5	4	9
0	1	2	3	4	5	6

A:

3	5	7	6	8	4	9
0	1	2	3	4	5	6

Sort the pink entries

A:

3	5	4	6	8	7	9
0	1	2	3	4	5	6

Notice how it is now **almost sorted**

Now do a final round of insertion sort over the entire array

A:

3	4	5	6	7	8	9
0	1	2	3	4	5	6

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Shellsort

- We just did a shellsort for k=3
- In general:
  - Think of the array as an interleaving of k lists
  - Sort each list separately using insertion sort
  - Then sort the resulting entire array using a final pass of insertion sort

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Shellsort Passes and Gap Sequences



THE UNIVERSITY OF MELBOURNE

- For large files, start with larger ***k*** and then repeat with smaller ***k***s

- It is common to start from somewhere in the sequence 1, 4, 13, 40, 121, 364, 1093, … and work backwards.
  - what is the sequence?

- For example, for an array of size 20,000, start by 364-sorting, then 121-sort, then 40-sort, and so on.

- Sequences with smaller gaps (a factor of about 2.3) appear to work better, but nobody really understands why.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Shellsort

- Fewer comparisons than insertion sort. Known to be worst-case $O(n\sqrt{n})$ for good gap sequences.

- Conjectured to be $O(n^{1.25})$ but the algorithm is very hard to analyse.

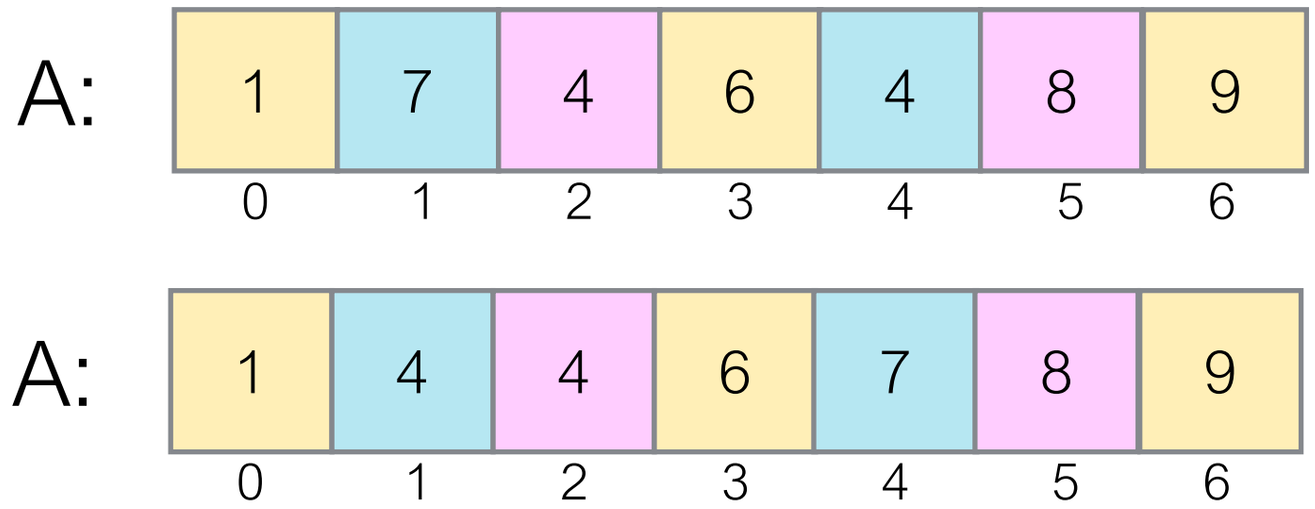
- Very good on medium-sized arrays (up to size 10,000 or so).

- In-place? yes

- Stable? ?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Shellsort: Stability



after sorting the blues
relative order of the two 4s has changed!

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Properties of Shellsort



THE UNIVERSITY OF MELBOURNE

- Fewer comparisons than insertion sort. Known to be worst-case $O(n\sqrt{n})$ for good gap sequences.

- Conjectured to be $O(n^{1.25})$ but the algorithm is very hard to analyse.

- Very good on medium-sized arrays (up to size 10,000 or so).

- In-place? yes

- Stable? no

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Other Instances of Decrease-and-Conquer by a Constant



THE UNIVERSITY OF MELBOURNE

- Insertion sort is a simple instance of the “decrease-and-conquer by a constant” approach.
- Another is the approach to topological sorting that repeatedly removes a source.
- In the next lecture we look at examples of “decrease by some factor”, leading to methods with logarithmic time behaviour or better!

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License